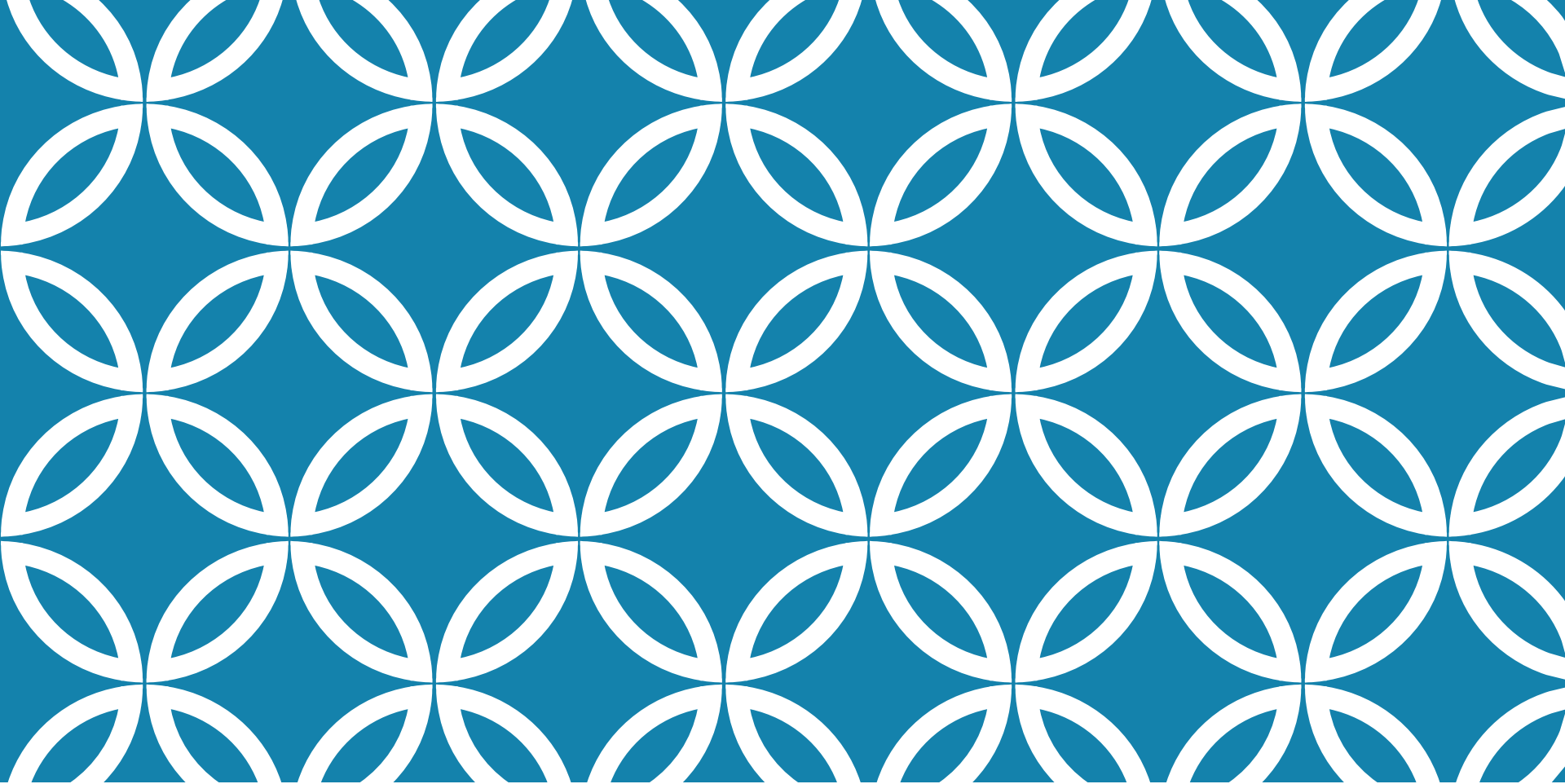


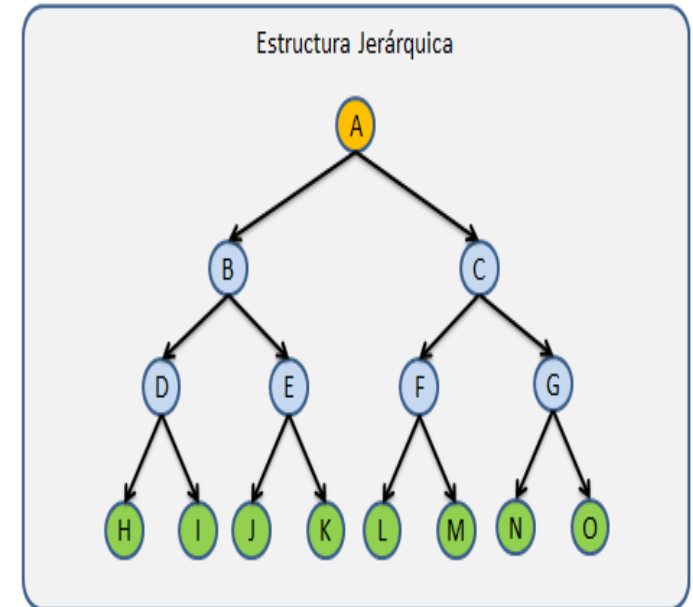


ESTRUCTURA DE DATOS

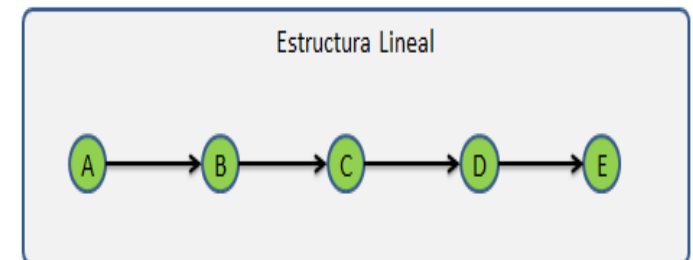
Primer Semestre 2022

ESTRUCTURA ÁRBOL

- Los árboles, a diferencia de las listas, son una estructura de datos no lineal, similar a una estructura de tipo jerárquica.
- Los árboles son estructuras de datos empleadas en informática, tanto para resolver problemas de hardware como de software, como por ejemplo en árboles de directorios. De igual manera cumplen un papel importante en aplicaciones para la toma de decisiones, como es el caso de un árbol de decisiones.
- También se emplean en programas para analizar circuitos eléctricos y para representar la estructura de fórmulas matemáticas, así como para organizar la información de bases de datos, para representar la estructura sintáctica de un programa fuente en compiladores y para la toma de decisiones.

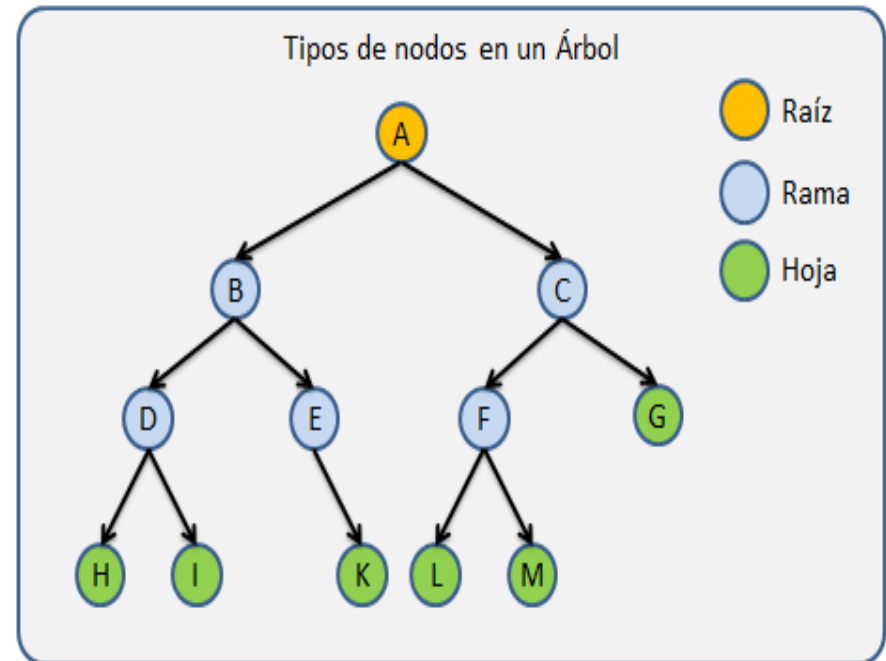


V.S.



ESTRUCTURA ÁRBOL

- ❑ Un **árbol** es una estructura de datos compuesta de un conjunto finito de datos denominados **nodos** y de un conjunto finito de líneas dirigidas, denominadas ramas, que conectan los nodos.
- ❑ Un árbol que no tiene ningún nodo se llama **árbol vacío o nulo**.
- ❑ Un **árbol** que no está vacío consta de un nodo raíz y potencialmente muchos niveles de nodos adicionales que forman una jerarquía.
- ❑ Los árboles se caracterizan por disponer sus nodos en forma jerárquica y no en forma lineal.

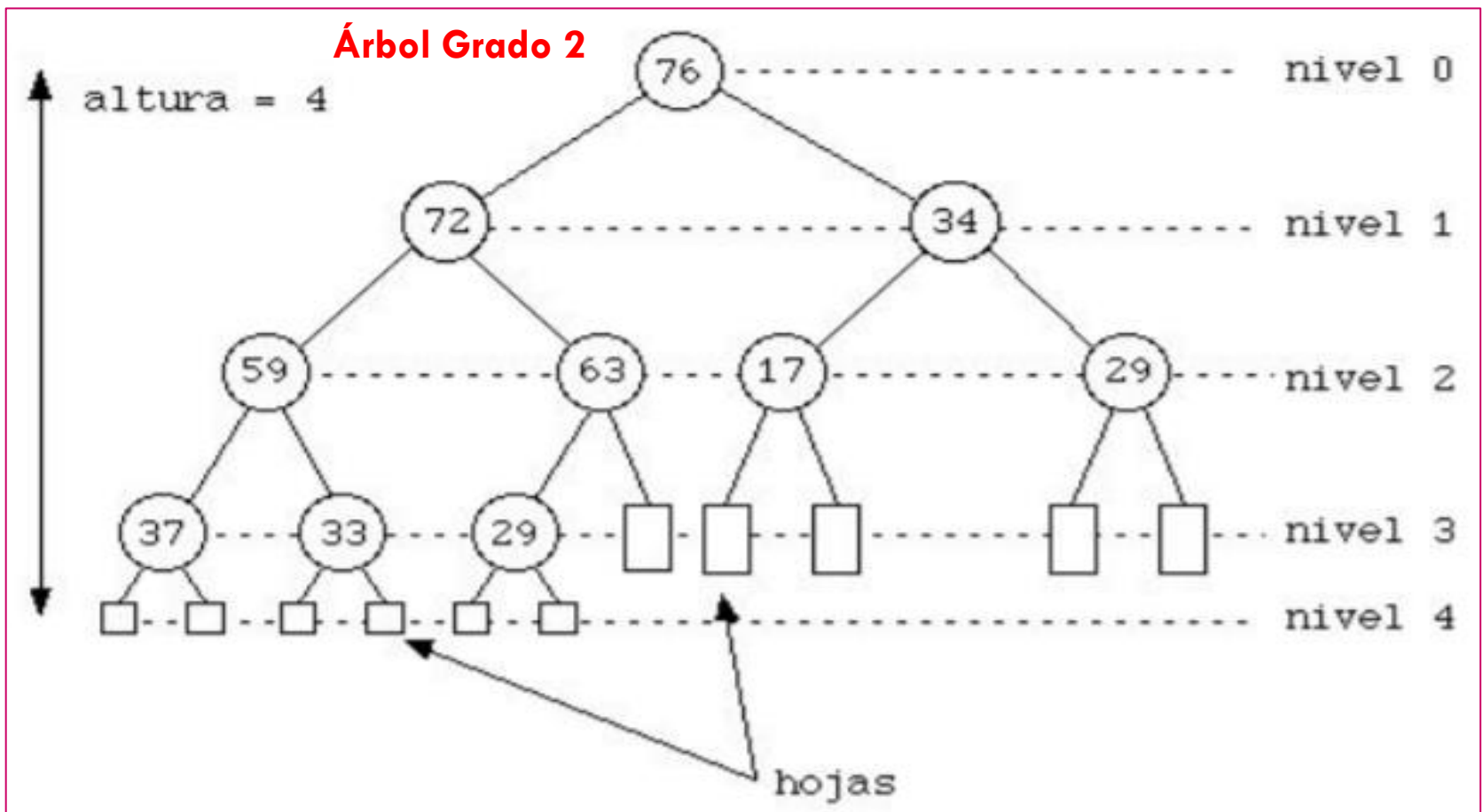


UN ÁRBOL TIENE

GRADO – ALTURA – PROFUNDIDAD – NIVEL

- ❑ **Grado:** Corresponde al número de hijos que tiene el árbol, en el nodo con más hijos.
- ❑ **Altura:** La altura es la cantidad de niveles.
- ❑ **Profundidad:** Se define como la longitud del camino (único) que comienza en la raíz y termina en el nodo. También se denomina nivel.
- ❑ **Nivel:** Se define para cada elemento del árbol, como la distancia a la raíz medida en nodos. El nivel de la raíz siempre será cero y el de sus hijos uno y así sucesivamente.

ÁRBOL: GRADO – ALTURA – PROFUNDIDAD - NIVEL



ÁRBOL BINARIO

Qué es un árbol binario?

Es un conjunto finito de elementos, denominados **nodos** y un conjunto finito de líneas dirigidas, denominadas **ramas**, que conectan los nodos.

Un **árbol binario** es un árbol en el que ningún nodo puede tener más de dos subárboles.

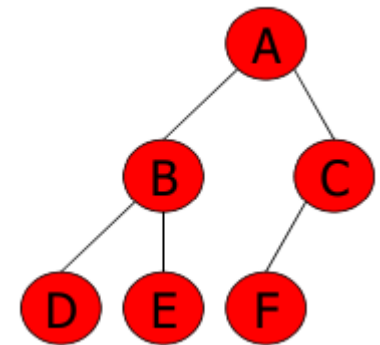
En un árbol binario cada nodo puede tener cero, uno o dos hijos (subárboles).

Se conoce el nodo de la izquierda como **hijo izquierdo** y el nodo de la derecha como **hijo derecho**.

Su estructura se organiza formando jerarquías: **PADRES** e **HIJOS**.

Los elementos de un árbol se llaman **NODOS**.

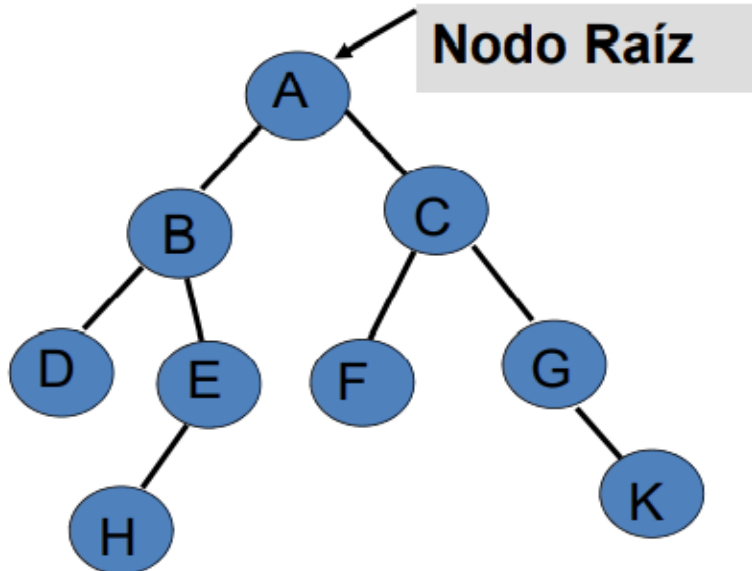
Si un nodo A tiene un enlace con un nodo B, entonces, A es el padre y B es el hijo.



- A es Padre.
- B y C son hijos de A.
- B es Padre.
- D y E hijos de B.
- F hijo de C.

ÁRBOL BINARIO:

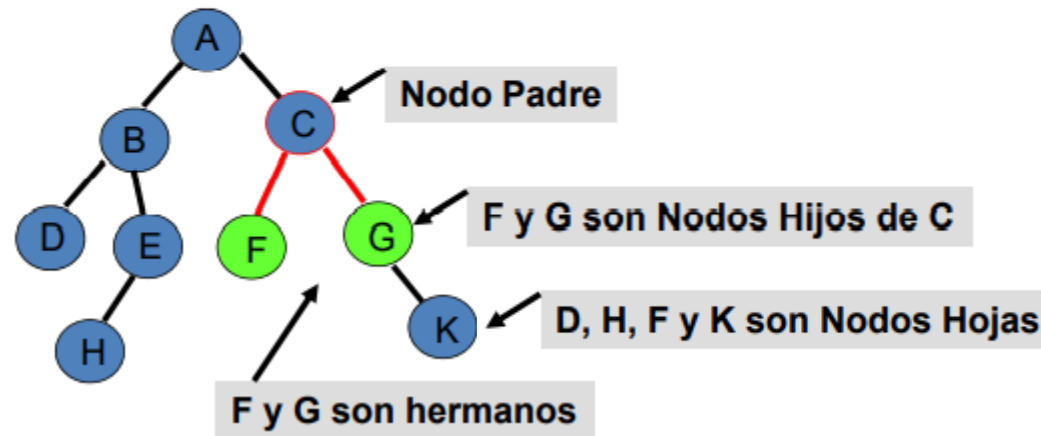
- Nodo: Cada elemento en un árbol.
- Nodo Raíz: Primer elemento agregado al árbol.



La **RELACIÓN**
entre los
elementos del
árbol es de
uno a muchos.

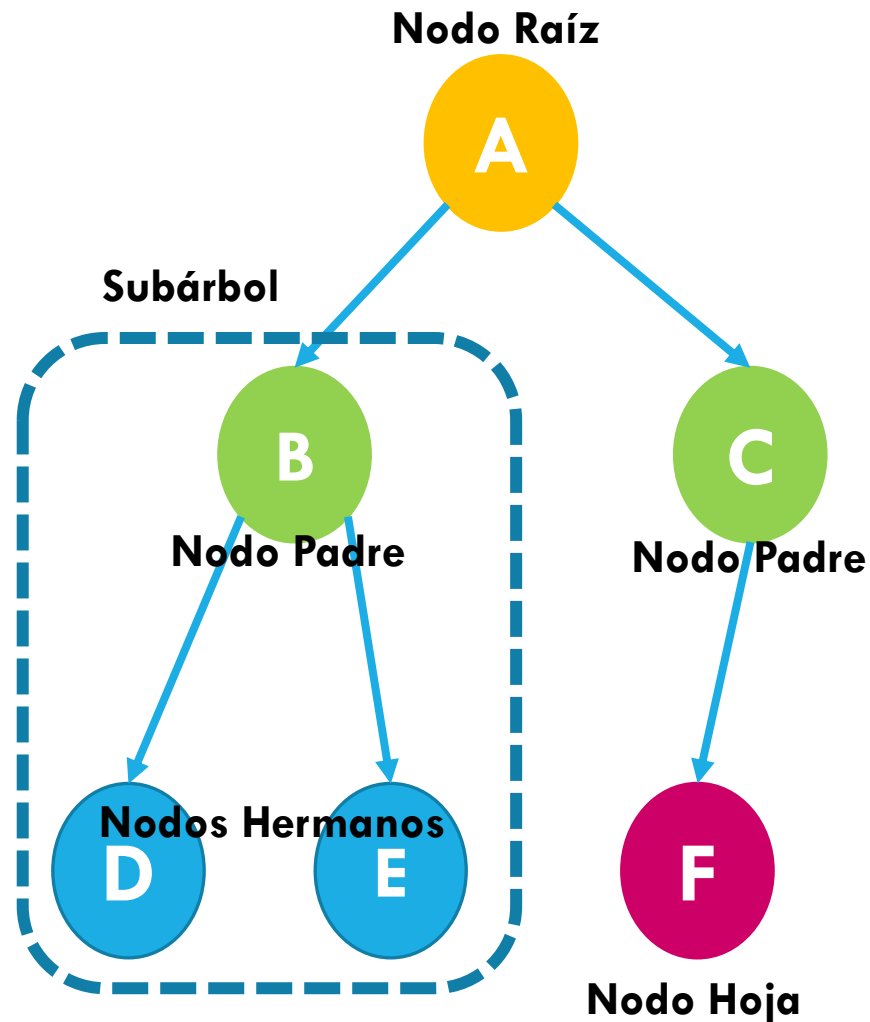
ÁRBOL BINARIO:

- Nodo Padre: Se le llama así al nodo predecesor de un elemento.
- Nodo Hijo: Es el nodo sucesor de un elemento.
- Hermanos: Nodos que tienen el mismo nodo padre.



- Nodo Hoja: Aquel nodo que no tiene hijos.

ÁRBOL BINARIO

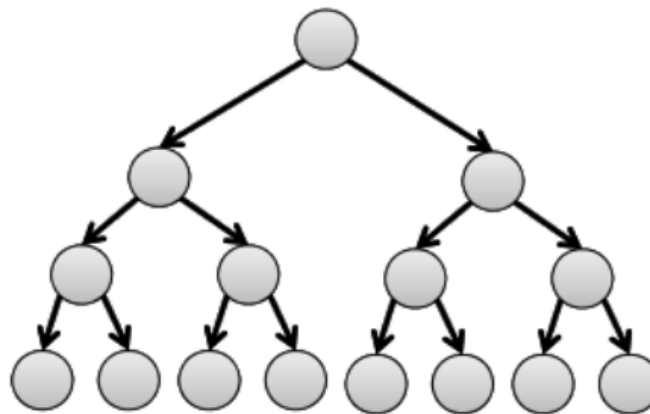
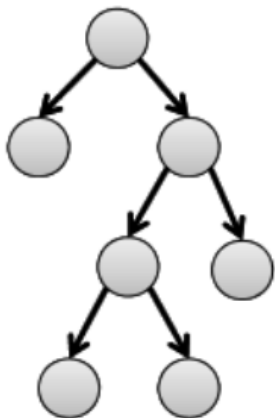


- Los hijos de un mismo padre se llaman: **hermanos**.
- Todos los nodos tienen al menos un padre, menos la raíz.
- Si los elementos no tienen hijos se llaman **hojas**.
- Un **subárbol** de un árbol, es cualquier nodo del árbol junto con todos sus descendientes.

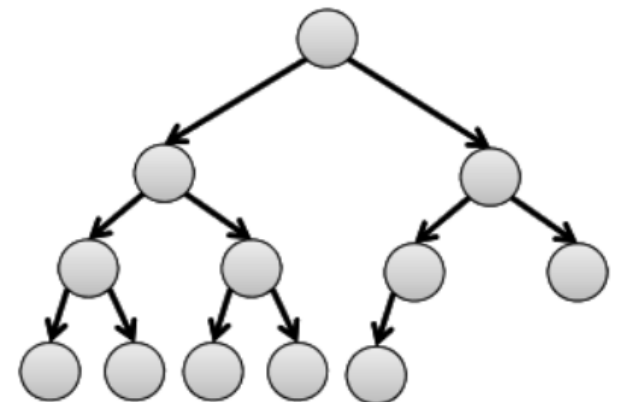
- A es **Padre**. Los nodos B y C son hijos de A. El nodo F es **hijo** de C.
- B es **Padre**. Los nodos D y E son sus hijos. D y E son **hermanos**.
- D, E y F son **hojas**.

VARIANTES DE UN ÁRBOL BINARIO

- 1.-Árbol estricto:** Si un subárbol está vacío, el otro también. Cada nodo puede tener 0 ó 2 hijos.
- 2.-Árbol lleno :** Árbol estricto donde en cada nodo la altura del subárbol izquierdo es igual a la del derecho, y ambos subárboles son árboles llenos.
- 3.-Árbol completo:** Árbol lleno hasta el penúltimo nivel. En el último nivel los nodos están agrupados a la izquierda.



2.- Árbol lleno



3.- Árbol completo

RECORRIDO DE UN ÁRBOL BINARIO

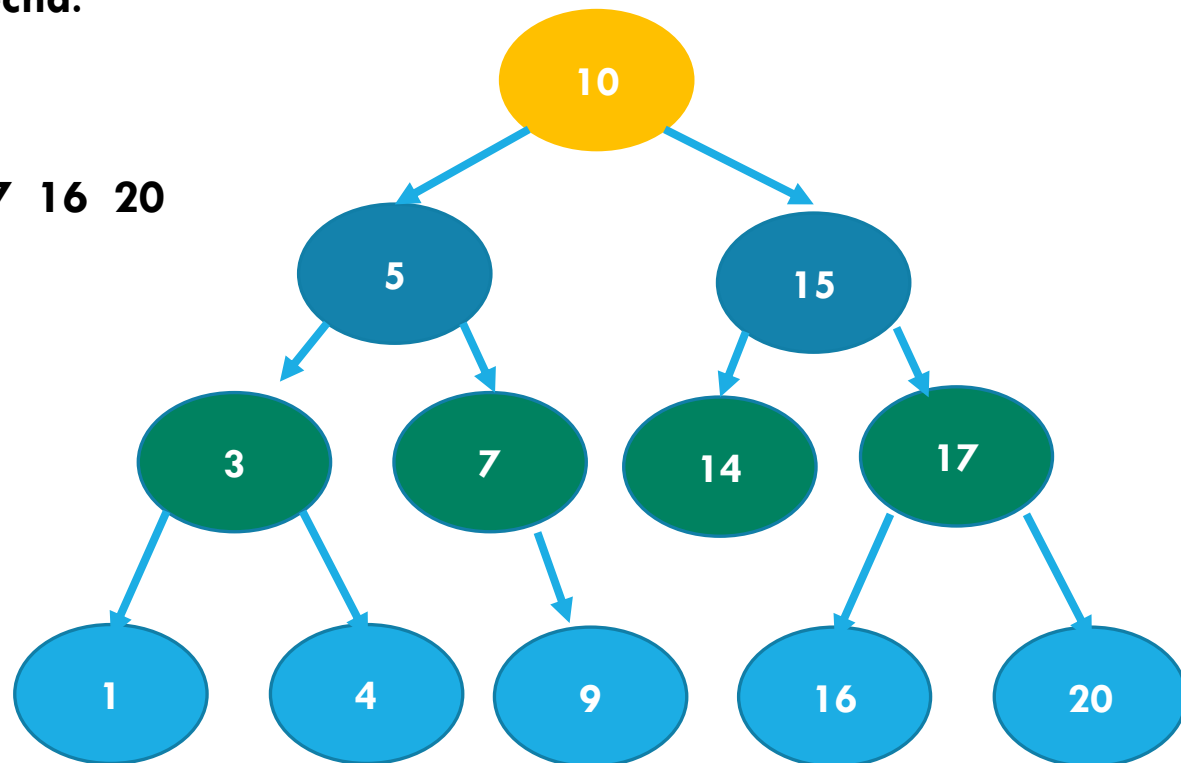


RECORRIDO PRE-ORDEN

Recorrer un árbol en preorden consiste en primer lugar, examinar el dato del nodo raíz, posteriormente se recorrer el subárbol izquierdo en preorden y finalmente se recorre el subárbol derecho en preorden. Esto significa que para cada subárbol se debe conservar el recorrido en preorden, **primero la raíz, luego la parte izquierda y posteriormente la parte derecha.**

Recorrido en pre-orden:

10 5 3 1 4 7 9 15 14 17 16 20

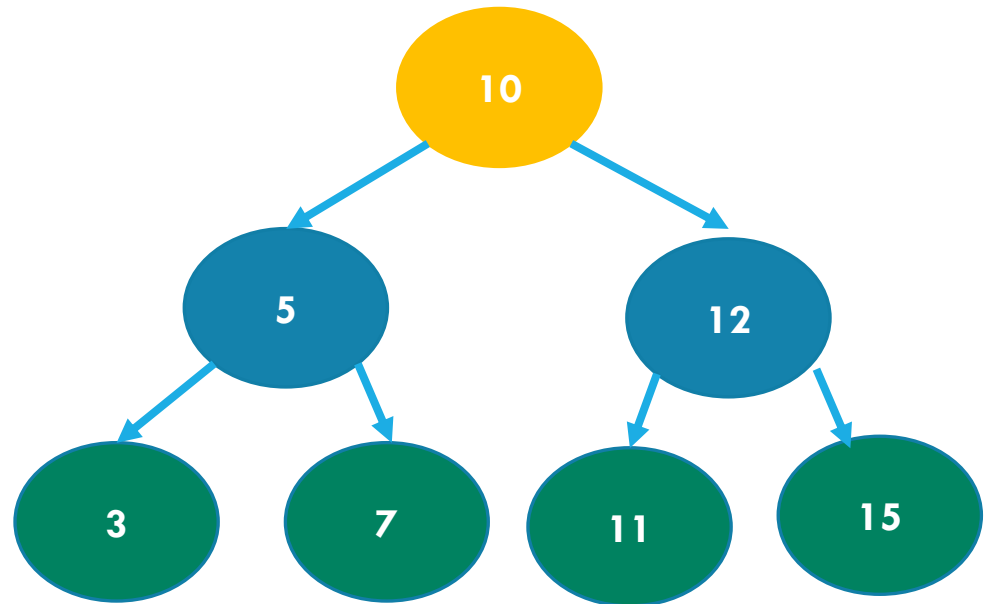


EJEMPLO POST-ORDEN

Recorrer un árbol en Postorden consiste en primer lugar en recorrer el subárbol izquierdo en Postorden, luego se recorre el subárbol derecho en Postorden y finalmente se visita el nodo raíz. Esto significa que para cada subárbol se debe conservar el recorrido en Postorden, es decir, primero **se visita la parte izquierda, luego la parte derecha y por último la raíz.**

Recorrido en post-orden:

3 7 5 11 15 12 10

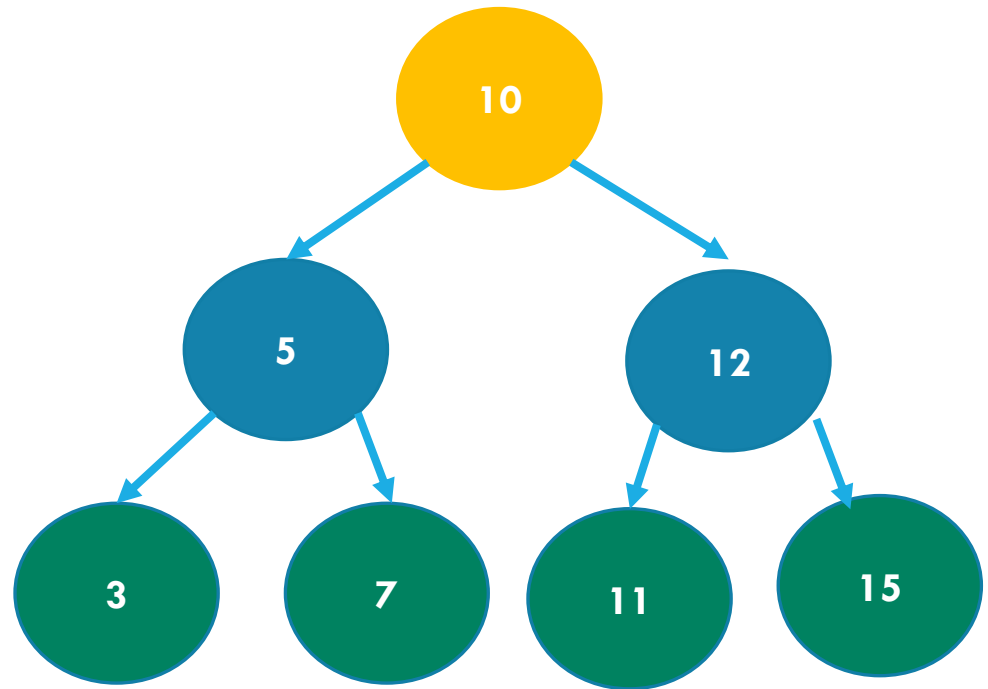


RECORRIDO INORDEN

Recorrer un árbol en Inorden consiste en primer lugar en recorrer el subárbol izquierdo en Inorden, luego se examina el dato del nodo raíz, y finalmente se recorre el subárbol derecho en Inorden. Esto significa que para cada subárbol se debe conservar el recorrido en Inorden, es decir, **primero se visita la parte izquierda, luego la raíz y posteriormente la parte derecha.**

Recorrido en inorden:

3 5 7 10 11 12 15



RECORDEMOS QUÉ ES UN TDA

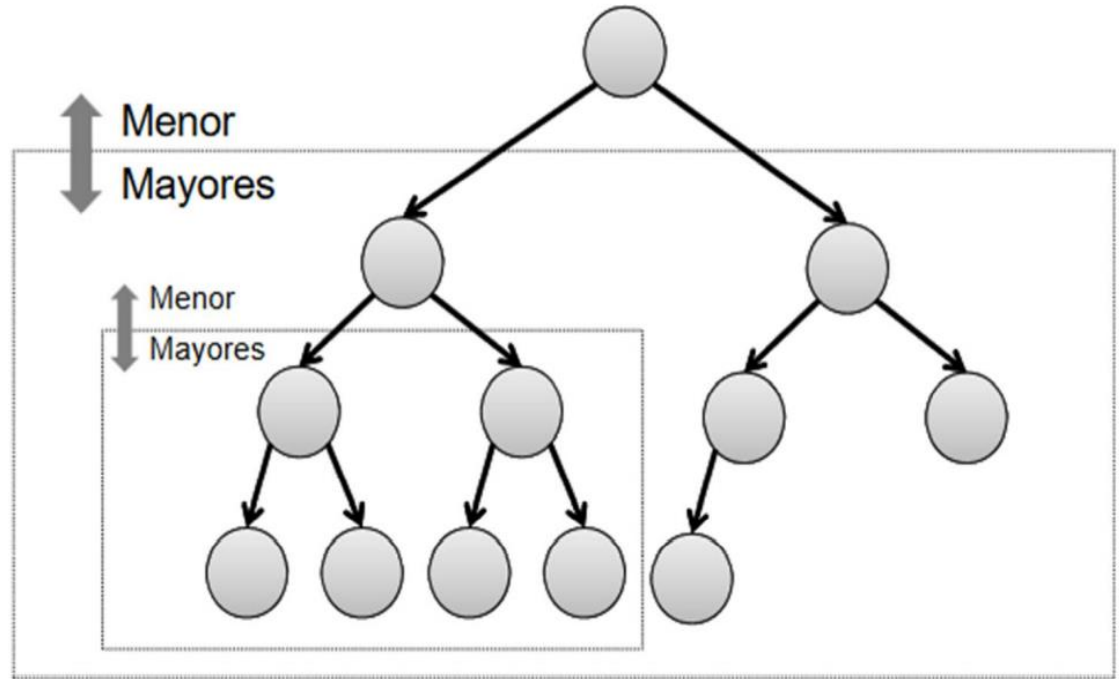
Un **Tipo de Dato Abstracto (TDA)** es un modelo constituido **por un conjunto de objetos y una colección de operaciones** realizables sobre él. Su representación como tipo concreto ha sido **abstraída** y a cuyos datos sólo se puede acceder a través de un conjunto de operaciones.

El **TDA** provee de una interfaz con la cual es posible realizar las operaciones permitidas, **abstrayéndose** de la manera en como estén implementadas dichas operaciones. Las **operaciones básicas**, por lo general, son **Inserción, Eliminación, Búsqueda, Vaciado e Inicialización**.

Las operaciones en un **TAD** deben ser cerradas, es decir, sólo deben acceder a los datos a través de las operaciones.

Algunos ejemplos de **TDA** son las **Pilas, Colas y Colas de Prioridad (Heap)**.

- El **HEAP** es el caso más notable de **cola de prioridad** y se define como un **árbol binario** con todos sus niveles completos excepto, generalmente, el último donde todos los nodos están ajustados a la izquierda.
- Cada nodo en un **heap** tiene mayor prioridad que sus descendientes, de manera que el elemento de prioridad máxima se encuentra siempre en la raíz del árbol.

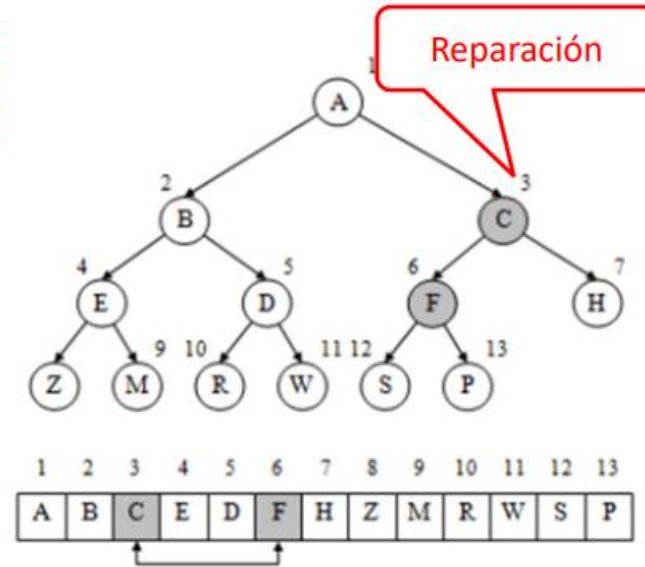
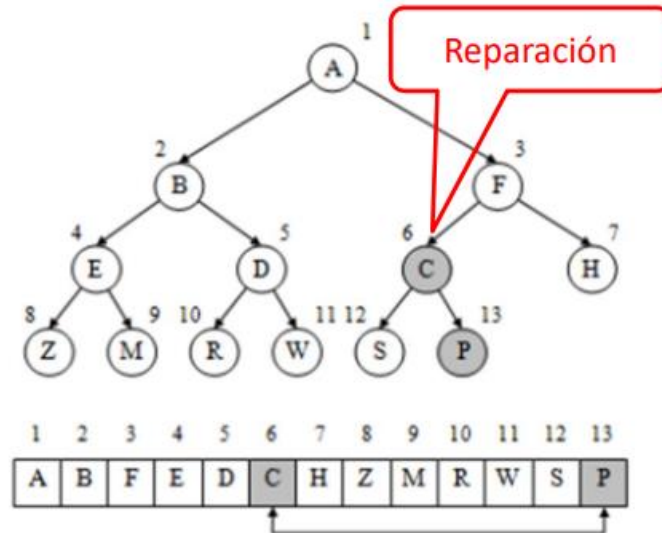
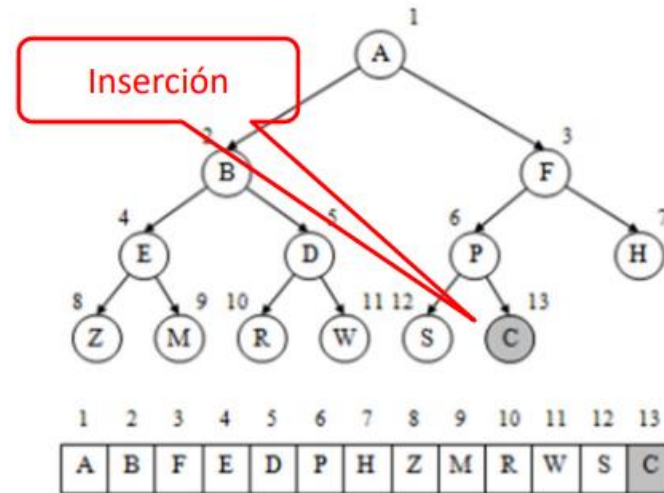
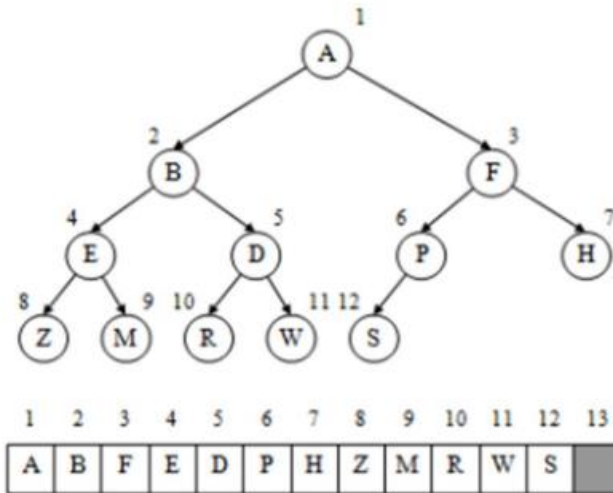


HEAP (MONTÍCULO)

PROPIEDADES

HEAP PROPIEDADES

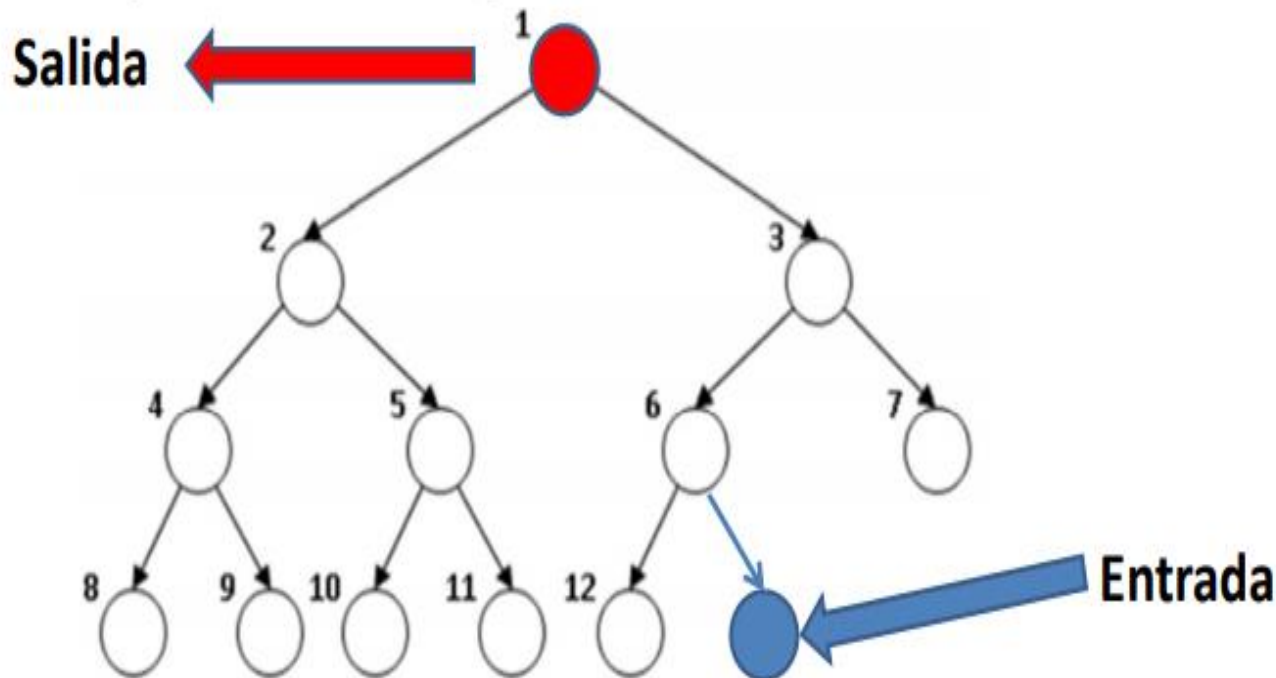
- El **nodo raíz** (en primera posición del vector) es el **mínimo (máxima prioridad)**.
- Si un sólo elemento no cumple la propiedad de heaps, es posible restablecer la propiedad mediante **ascensos** sucesivos en el árbol (intercambiándole con su padre) o mediante **descensos** en el árbol (intercambiándole con el mayor de sus hijos). El número de operaciones es proporcional a la **altura**.
- Para **insertar** un nuevo elemento se sitúa al final del vector (última hoja del árbol) y se **asciende** hasta que cumpla la propiedad.
- Para **eliminar la raíz** se intercambia con el último elemento (que se elimina en $O(1)$) y se **desciende** la nueva raíz hasta que cumpla la propiedad.



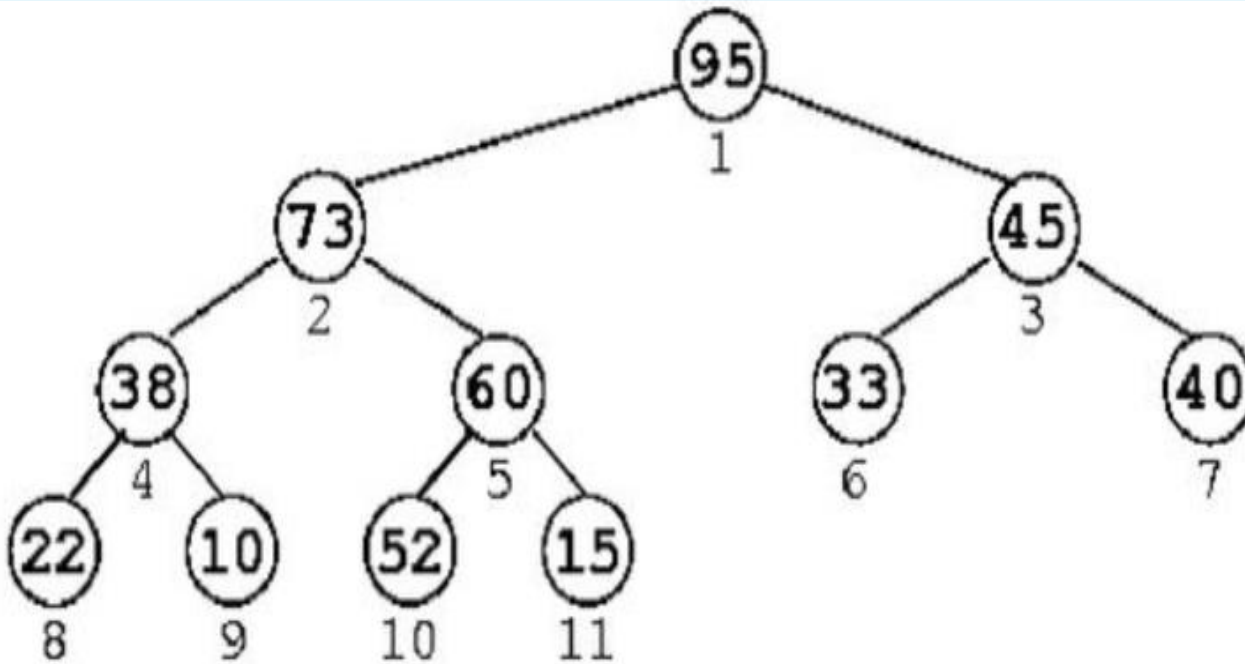
Ejemplo HEAP

HEAP

En un **HEAP** los elementos se ingresan por nivel, de izquierda a derecha. Después de un ingreso se debe reparar la eventual alteración de la propiedad de heap. Debido a la forma de organización del árbol, se puede usar un arreglo para representarlo. Basta con numerar los nodos consecutivamente por nivel, de arriba hacia abajo y de izquierda a derecha



HEAP

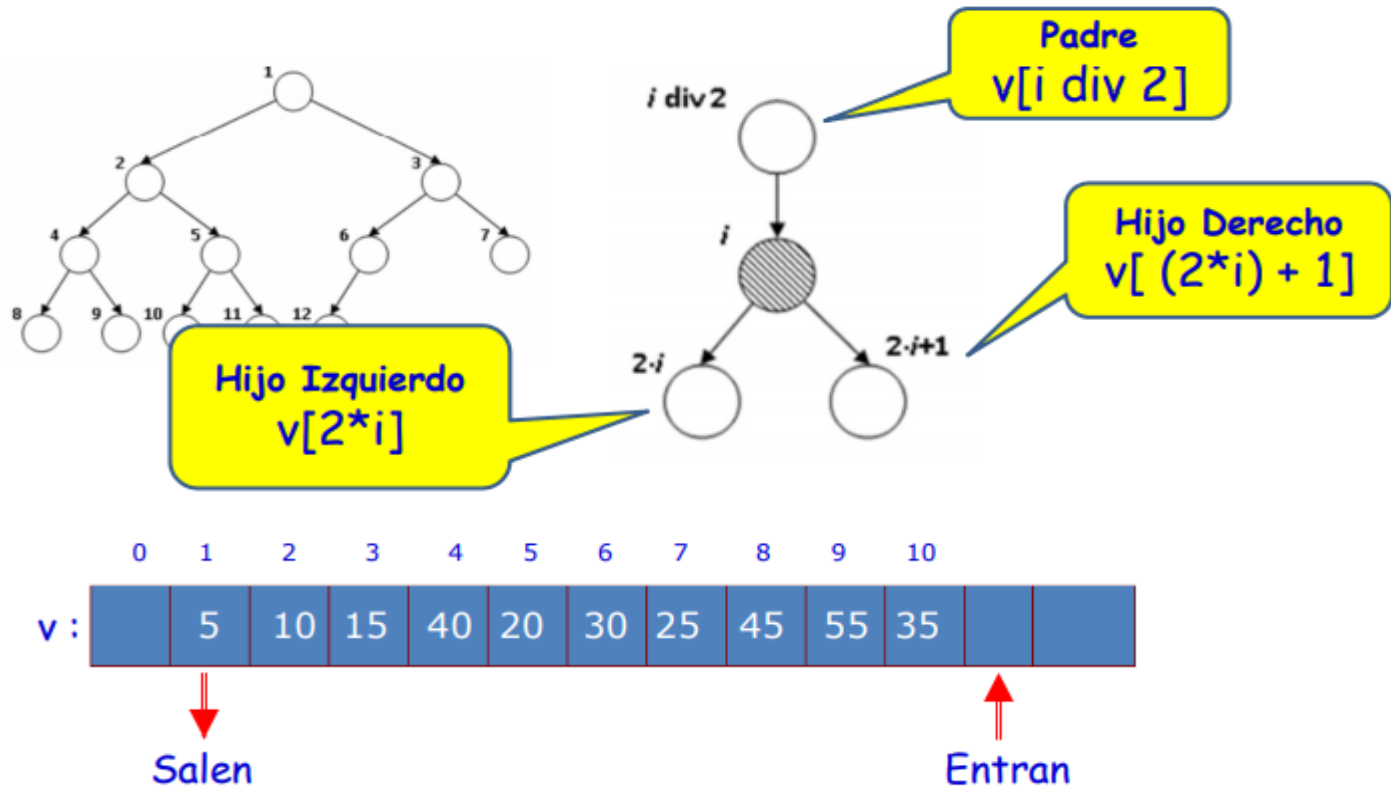


La numeración por niveles (indicada bajo cada nodo) son los subíndices en donde cada elemento sería almacenado en el arreglo.

95	73	45	38	60	33	40	22	10	52	15
1	2	3	4	5	6	7	8	9	10	11

HEAP

SIMULADOR DE HEAP XD



- El padre de un elemento $v[i]$ es $v[i]$, con $i=i/2$
- Un elemento $v[i]$ tiene hijos $v[i]$ y $v[i+1]$, con $i= 2 \cdot i$

COLA DE PRIORIDAD

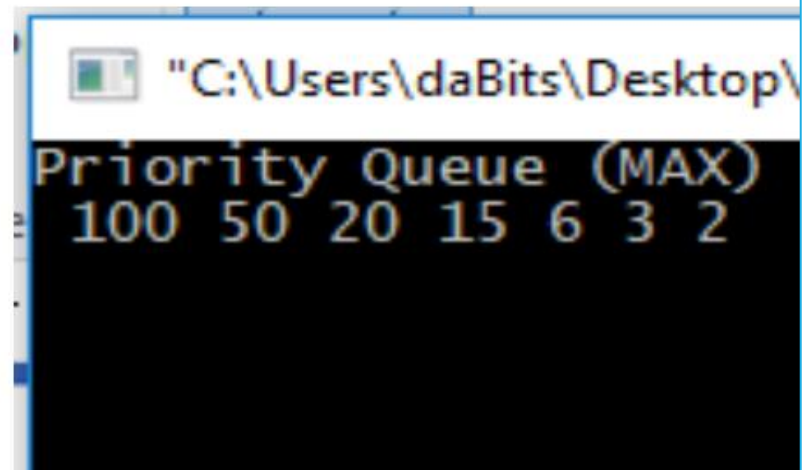
- Es una cola cuyos elementos tienen asociada una prioridad que determina el orden en que son extraídos.
- Una prioridad es un número entre 1 y p , donde 1 es la prioridad más alta.
- En una cola de prioridad, se puede agregar un elemento de cierta prioridad, o bien, extraer el elemento de máxima prioridad.

COLA DE PRIORIDAD

```
#include <iostream>
#include <bits/stdc++.h>
using namespace std;

int main() {

    cout << "Priority Queue (MAX)" << endl;
    priority_queue <int> h;
    h.push(2);
    h.push(20);
    h.push(50);
    h.push(3);
    h.push(6);
    h.push(100);
    h.push(15);
    while (!h.empty()) {
        cout << " " << h.top();
        h.pop();
    }
}
```



COLA DE PRIORIDAD

Definición: `priority_queue<Type, Container, Functional>`

- **Type** es el tipo de datos.
- **Container** es el tipo de contenedor. El contenedor debe ser un contenedor implementado con una matriz, como vector, deque, etc., pero no una lista. El valor predeterminado en STL es vector.
- **Functional** es la forma de comparación. Cuando necesite usar un tipo de datos personalizado, debe pasar estos tres parámetros. Cuando use tipos de datos básicos, solo necesita pasar el tipo de datos.

Ejemplo:

- ❖ `priority_queue <int,vector<int>,greater<int> > q; //` Cola ascendente
- ❖ `priority_queue <int,vector<int>,less<int> >q; //` Cola descendente

COLA DE PRIORIDAD

Operación básica:

- ❖ `empty ()` devuelve verdadero si la cola está vacía
- ❖ `pop ()` eliminar el elemento superior
- ❖ `push ()` agrega un elemento
- ❖ `size ()` devuelve el número de elementos en la cola de prioridad
- ❖ `top ()` devuelve el elemento superior de la cola de prioridad

EJEMPLOS HEAP

1

- Heap / cola de prioridad básico:
- <https://onlinegdb.com/Oe5QmMv2T>

2

- Heap / cola de prioridad ascendente:
- <https://onlinegdb.com/GWOobMzCv>

3

- Utilizando Heap para ordenar información de una clase Paciente
- <https://onlinegdb.com/XhlozBlgb>

4

- Clase Heap(string) implementada con vectores
- https://onlinegdb.com/vPXDjcu_H

EJEMPLOS COMBINADOS

- ❑ Colas utilizando clase. Cola creada a partir de la librería Queue.

<https://onlinegdb.com/C-s9-33P2>

- ❑ Validación de datos de entrada

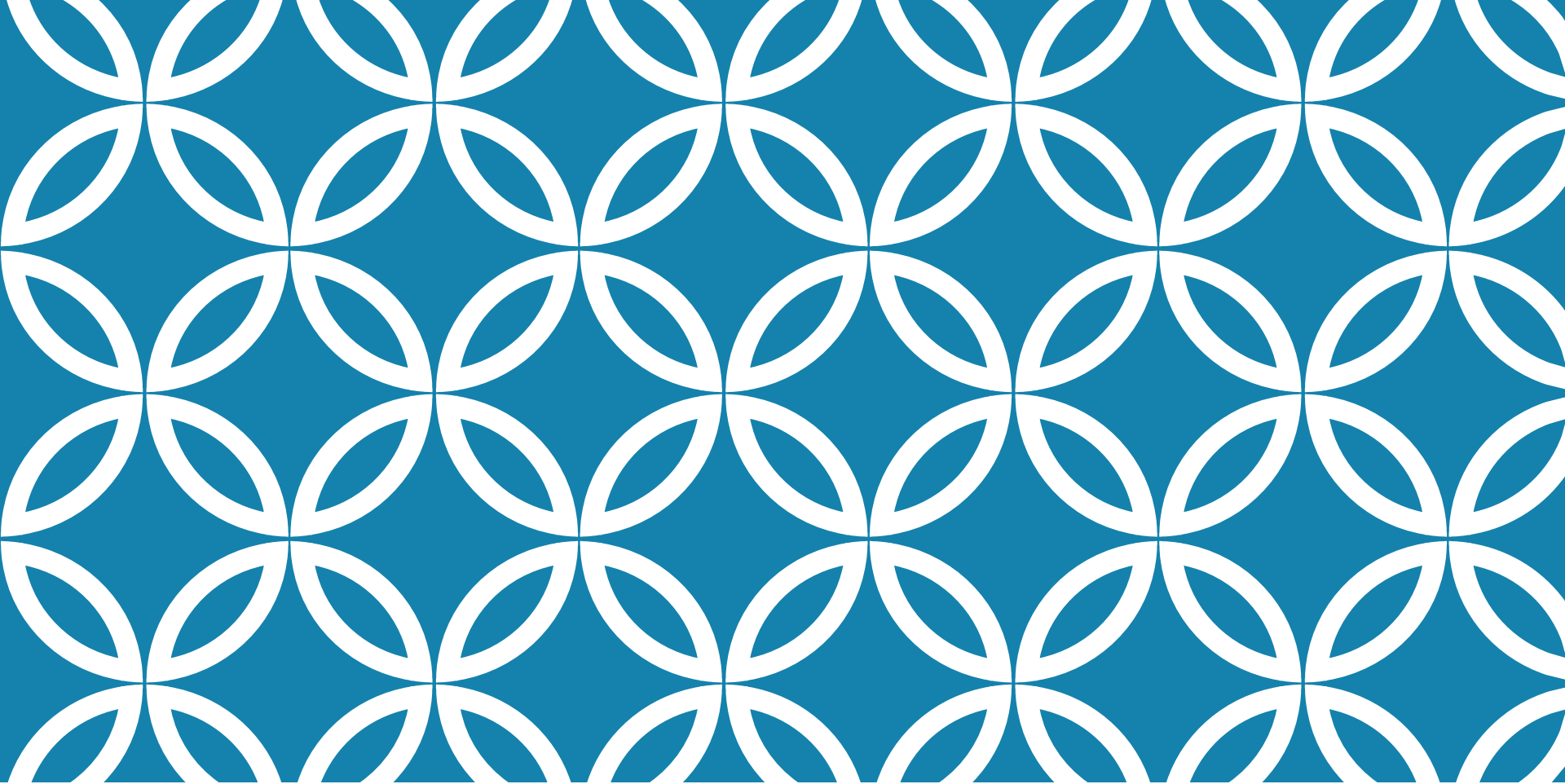
<https://onlinegdb.com/sWGeJJKop>

- ❑ Stack: verificación de una formula

https://onlinegdb.com/SgR5_8_NP

- ❑ Heap, Colas y Stack utilizando clase Persona

<https://onlinegdb.com/ZGPqjlfG>



ESTRUCTURA DE DATOS

Primer Semestre 2022



UNIVERSIDAD
TECNOLÓGICA
METROPOLITANA
UTEM del Estado de Chile